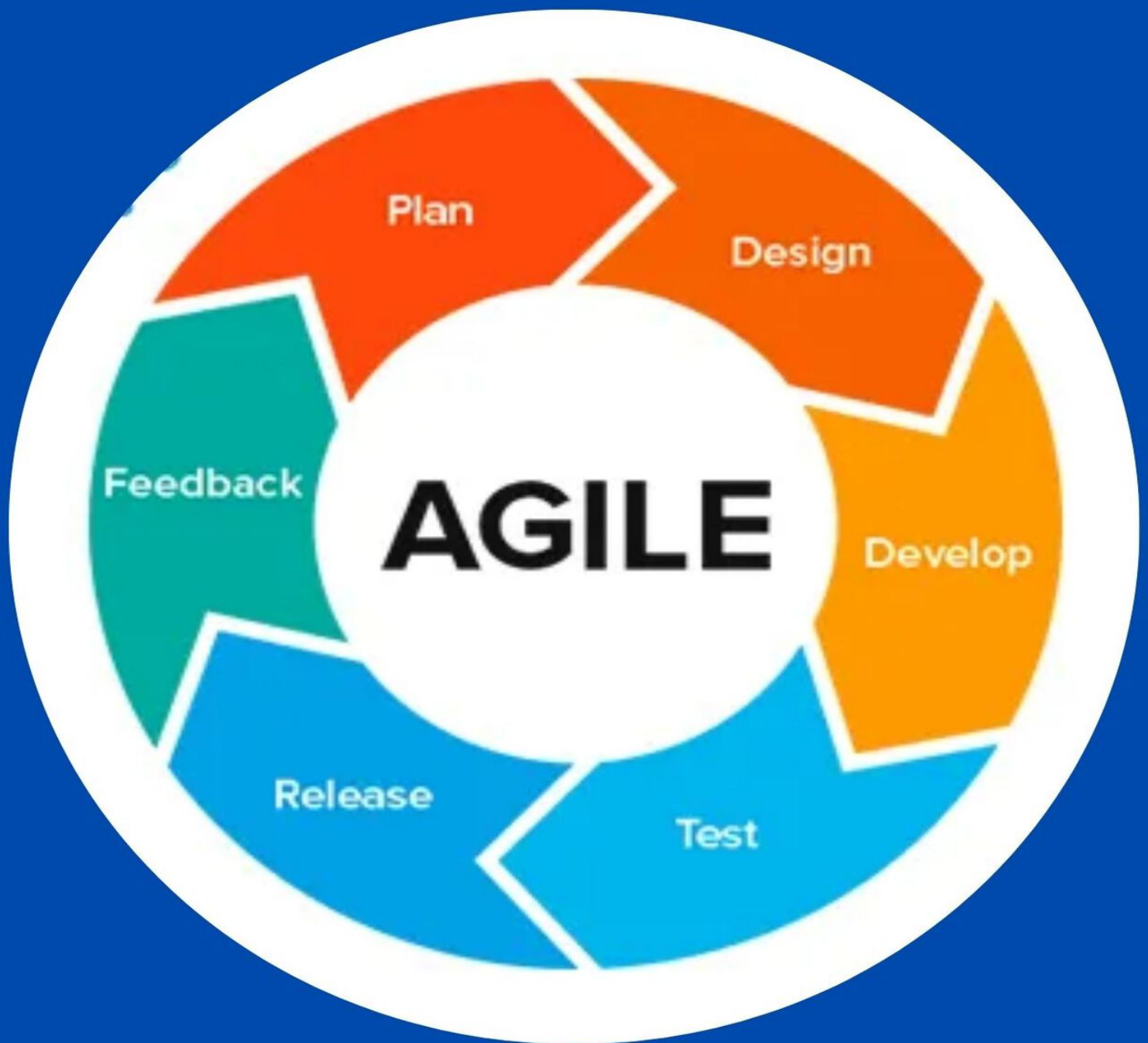


APRENDE PRUEBAS AGILES



CARMELO RAMOS

Tutorial de pruebas ágiles

Agile Testing es una práctica de prueba de software que sigue los principios del desarrollo de software ágil. Agile Testing involucra a todos los miembros del equipo del proyecto, con experiencia especial aportada por los probadores. La prueba no es una fase separada y está entrelazada con todas las fases de desarrollo, como los requisitos, el diseño y la codificación y la generación de casos de prueba. Las pruebas se realizan simultáneamente durante el ciclo de vida del desarrollo.

Audiencia

El público objetivo de este tutorial son los profesionales de pruebas de software, los expertos en calidad del software y los desarrolladores de software.

Prerrequisitos

Antes de continuar con este tutorial, debe tener un conocimiento básico del ciclo de vida del desarrollo de software (SDLC). Será beneficioso tener un conocimiento básico de las pruebas de software (manual o automatización).

Pruebas ágiles: descripción general

Agile es una metodología de desarrollo iterativa, donde tanto las actividades de desarrollo como las de prueba son concurrentes. La prueba no es una fase separada; La codificación y las pruebas se realizan de forma interactiva e incremental, lo que da como resultado un producto final de calidad que cumple con los requisitos del cliente. Además, la integración continua da como resultado la eliminación temprana de defectos y, por lo tanto, ahorros de tiempo, esfuerzo y costos.

Manifiesto ágil

El Manifiesto Agile fue publicado por un equipo de desarrolladores de software en 2001, destacando la importancia del equipo de desarrollo, acomodando los requisitos cambiantes y la participación del cliente.

El Manifiesto Ágil es:

Estamos descubriendo mejores formas de desarrollar software haciéndolo y ayudando a otros a hacerlo. A través de este trabajo, hemos llegado a valorar:

- Individuos e interacciones sobre procesos y herramientas.
- Software de trabajo sobre documentación completa.
- Colaboración con el cliente sobre negociación de contratos.
- Responde al cambio sobre el siguiente plan.

Es decir, si bien hay valor en los elementos de la derecha, valoramos más los elementos de la izquierda.

¿Qué son las pruebas ágiles?

Agile Testing es una práctica de prueba de software que sigue los principios del desarrollo de software ágil.

Agile Testing involucra a todos los miembros del equipo del proyecto, con experiencia especial aportada por los probadores. La prueba no es una fase separada y está entretrejida con todas las fases de desarrollo, como los requisitos, el diseño y la codificación y la generación de casos de prueba. Las pruebas se realizan simultáneamente durante el ciclo de vida del desarrollo.

Además, con la participación de los probadores en todo el ciclo de vida del desarrollo junto con los miembros del equipo multifuncional, la contribución de los probadores para construir el software según los requisitos del cliente, con un mejor diseño y código, sería posible.

Agile Testing cubre todos los niveles de testing y todos los tipos de testing.

Pruebas ágiles vs. Prueba de cascada

En una metodología de desarrollo en cascada, las actividades del ciclo de vida del desarrollo ocurren en fases que son secuenciales. Por lo tanto, la prueba es una fase separada y se inicia solo después de la finalización de la fase de desarrollo.

A continuación se muestran los aspectos más destacados de las diferencias entre las pruebas ágiles y las pruebas en cascada:

Pruebas ágiles	Prueba de cascada
La prueba no es una fase separada y ocurre al mismo tiempo que el desarrollo.	La prueba es una fase separada. Todos los niveles y tipos de pruebas pueden comenzar solo después de la finalización del desarrollo.
Los probadores y desarrolladores trabajan juntos.	Los probadores trabajan por separado de los desarrolladores.
Los evaluadores participan en la elaboración de requisitos. Esto ayuda a mapear los requisitos a los comportamientos en el escenario del mundo real y también a enmarcar los criterios de aceptación. Además, los casos de prueba de aceptación lógica estarían listos junto con los requisitos.	Los probadores pueden no participar en la fase de requisitos.
Las pruebas de aceptación se realizan después de cada iteración y se buscan los comentarios de los clientes.	Las pruebas de aceptación se realizan solo al final del proyecto.
Cada iteración completa sus propias pruebas, lo que permite implementar pruebas de regresión cada vez que se lanzan nuevas funciones o lógica.	Las pruebas de regresión se pueden implementar solo

	después de completar el desarrollo.
Sin demoras entre la codificación y la prueba.	Retrasos habituales entre la codificación y la prueba.
Prueba continua con niveles de prueba superpuestos.	La prueba es una actividad cronometrada y los niveles de prueba no pueden superponerse.
Las pruebas son una buena práctica.	Las pruebas a menudo se pasan por alto.

Principios de prueba ágiles

Los principios de las pruebas ágiles son:

-

Las pruebas hacen avanzar el proyecto : las pruebas continuas son la única forma de garantizar un progreso continuo. Agile Testing proporciona retroalimentación de forma continua y el producto final cumple con las demandas comerciales.

-

La prueba no es una fase : el equipo ágil prueba junto con el equipo de desarrollo para garantizar que las funciones implementadas durante una iteración determinada se realicen. Las pruebas no se guardan para una fase posterior.

-

Todos prueban : en las pruebas ágiles, todo el equipo, incluidos analistas, desarrolladores y probadores, prueba la aplicación. Después de cada iteración, incluso el cliente realiza la prueba de aceptación del usuario.

-

Acortar los ciclos de retroalimentación : en Agile Testing, el equipo empresarial llega a conocer el desarrollo del producto para todas y cada una de las iteraciones. Están involucrados en cada iteración. La retroalimentación continua acorta el tiempo de respuesta de la retroalimentación y, por lo tanto, el costo involucrado en solucionarlo es menor.

-

Mantenga limpio el código : los defectos se corrigen a medida que surgen dentro de la misma iteración. Esto asegura un código limpio en cualquier hito del desarrollo.

-

Documentación liviana : en lugar de documentación de prueba completa, probadores ágiles:

-

- Utilice listas de verificación reutilizables para sugerir pruebas.

-

- Concéntrese en la esencia de la prueba en lugar de los detalles incidentales.

-

- Utilice estilos / herramientas de documentación ligeros.

-

- Capture ideas de prueba en cartas para pruebas exploratorias.

-

- Aproveche los documentos para múltiples propósitos.

-

Aprovechamiento de un artefacto de prueba para pruebas manuales y automatizadas : el mismo artefacto de secuencia de comandos de prueba se puede utilizar para pruebas manuales y como entrada para

pruebas automatizadas. Esto elimina el requisito de documentación de prueba manual y luego un script de prueba de automatización equivalente.

-

"Listo Listo", no solo hecho : en Agile, se dice que una función no se realiza después del desarrollo, sino después del desarrollo y las pruebas.

-

Test-Last vs. Test Driven : los casos de prueba se escriben junto con los requisitos. Por lo tanto, el desarrollo puede impulsarse mediante pruebas. Este enfoque se denomina Desarrollo impulsado por pruebas (TDD) y Desarrollo impulsado por pruebas de aceptación (ATDD). Esto contrasta con las pruebas como última fase en Waterfall Testing.

Actividades de prueba ágiles

Las actividades de pruebas ágiles a nivel de proyecto son:

-

Planificación de lanzamientos (plan de prueba)

-

Para cada iteración,

-

Actividades de prueba ágiles durante una iteración

-

Pruebas de regresión

-

Actividades de lanzamiento (relacionadas con la prueba)

Las actividades de prueba ágiles durante una iteración incluyen:

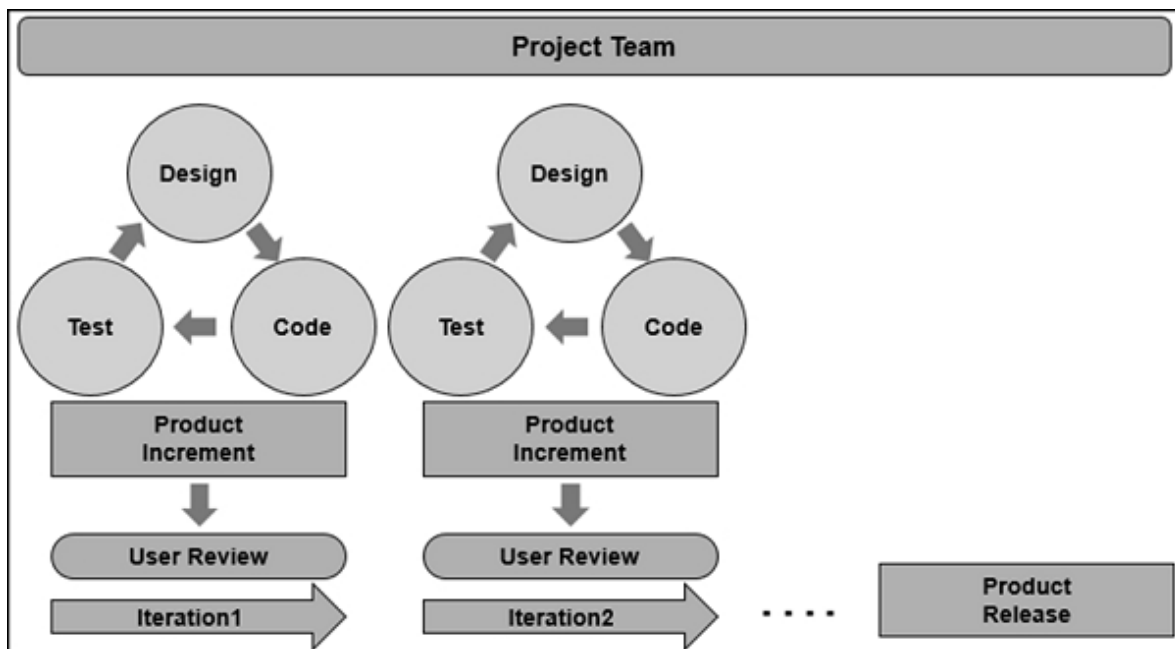
- Participar en la planificación de iteraciones
- Estimación de tareas desde el punto de vista de las pruebas
- Escribir casos de prueba utilizando las descripciones de funciones

- Examen de la unidad
- Pruebas de integración
- Prueba de funciones
- Reparación de defectos
- Pruebas de integración
- Test de aceptación
- Informe de estado sobre el progreso de las pruebas
- Seguimiento de defectos

Pruebas ágiles: metodologías

Agile es una metodología de desarrollo iterativa, donde todo el equipo del proyecto participa en todas las actividades. Los requisitos evolucionan a medida que avanzan las iteraciones, a través de la colaboración entre el cliente y los equipos autoorganizados. Como la codificación y las pruebas se realizan de forma interactiva e incremental, durante el curso del desarrollo, el producto final sería de calidad y garantizaría los requisitos del cliente.

Cada iteración da como resultado un incremento de producto de trabajo integrado y se entrega para las pruebas de aceptación del usuario. La retroalimentación del cliente así obtenida sería una entrada para las siguientes / subsiguientes iteraciones.



Integración continua, calidad continua

La integración continua es la clave para el éxito del desarrollo ágil. Integre con frecuencia, al menos diariamente, de modo que esté listo para un lanzamiento cuando sea necesario. Las pruebas en Agile se convierten en un componente esencial de todas las fases del desarrollo, asegurando la calidad continua del producto. La retroalimentación constante de todos los involucrados en el proyecto se suma a la calidad del producto.

En Agile, se le da la máxima importancia a la comunicación y las solicitudes de los clientes se reciben cuando es necesario. Esto le da la satisfacción al cliente de que todos los insumos son considerados y el producto de calidad de trabajo está disponible durante todo el desarrollo.

Metodologías ágiles

Hay varias metodologías ágiles que apoyan el desarrollo ágil. Las metodologías ágiles incluyen:

Melé

Scrum es un método de desarrollo ágil que enfatiza el enfoque centrado en el equipo. Aboga por la participación de todo el equipo en todas las actividades de desarrollo del proyecto.

XP

eXtreme Programming se centra en el cliente y se centra en los requisitos en constante cambio. Con lanzamientos frecuentes y comentarios de los clientes, el producto final será de calidad y cumplirá con los requisitos del cliente que se aclararán durante el proceso.

Cristal

Crystal se basa en fletamento, entrega cíclica y cierre.

-

El fletamento implica formar un equipo de desarrollo, realizar un análisis preliminar de viabilidad, llegar a un plan inicial y la metodología de desarrollo.

-

La entrega cíclica con dos o más ciclos de entrega se centra en la fase de desarrollo y la entrega final del producto integrado.

-

Durante el cierre, se realizan las revisiones y reflexiones posteriores a la implementación en el entorno del usuario.

FDD

El desarrollo basado en características (FDD) implica el diseño y la construcción de características. La diferencia entre FDD y otras metodologías de desarrollo ágil es que las características se desarrollan en fases específicas y cortas por separado.

DSDM

El método de desarrollo de software dinámico (DSDM) se basa en el desarrollo rápido de aplicaciones (RAD) y está alineado con el marco ágil. DSDM se enfoca en la entrega frecuente del producto, involucrando activamente a los usuarios y capacitando a los equipos para que tomen decisiones rápidas.

Desarrollo de software ajustado

En Lean Software Development, el foco está en eliminar el desperdicio y dar valor al cliente. Esto da como resultado un rápido desarrollo y un producto de valor.

El desperdicio incluye trabajo parcialmente realizado, trabajo irrelevante, características que no son utilizadas por el cliente, defectos, etc. que se suman a demoras en la entrega.

Los **Principios Lean** son:

- Eliminar residuos

- Amplificar el aprendizaje
- Compromiso de retraso
- Empoderar al equipo
- Entrega Rápido
- Construya integridad en
- Ver el todo

Kanban

Kanban se enfoca en administrar el trabajo con énfasis en la entrega justo a tiempo (JIT), sin sobrecargar a los miembros del equipo. Las tareas se muestran para que las vean todos los participantes y para que los miembros del equipo extraigan el trabajo de una cola.

Kanban se basa en:

- Tablero Kanban (visual y persistente en todo el desarrollo)
- Límite de trabajo en curso (WIP)
- Tiempo de espera

Metodologías de prueba ágiles

Las prácticas de prueba están bien definidas para cada proyecto, sea Agile o no, para entregar productos de calidad. Los principios de las pruebas tradicionales se utilizan con bastante frecuencia en las pruebas ágiles. Uno de ellos es Early Testing que se centra en:

- Redacción de casos de prueba para expresar el comportamiento del sistema.
- Prevención, detección y eliminación tempranas de defectos.
- Asegurarse de que se ejecuten los tipos de prueba correctos en el momento correcto y como parte del nivel de prueba correcto.

En todas las metodologías ágiles que discutimos, las pruebas ágiles en sí mismas son una metodología. En todos los enfoques, los casos de prueba se escriben antes que la codificación.

En este tutorial, nos centraremos en Scrum como la metodología de pruebas ágiles.

Las otras metodologías de prueba ágiles comúnmente utilizadas son:

- **Desarrollo** basado en pruebas (TDD): el desarrollo basado en pruebas (TDD) se basa en la codificación guiada por pruebas.
- **Desarrollo impulsado por pruebas de aceptación (ATDD)** : el desarrollo impulsado por pruebas de aceptación (ATDD) se basa en la comunicación entre los clientes, desarrolladores y probadores y está impulsado por criterios de aceptación predefinidos y casos de prueba de aceptación.
- **Desarrollo impulsado por el comportamiento (BDD)**: las pruebas de Desarrollo impulsado por el comportamiento (BDD) se basan en el comportamiento esperado del software que se está desarrollando.

Ciclo de vida de pruebas ágiles

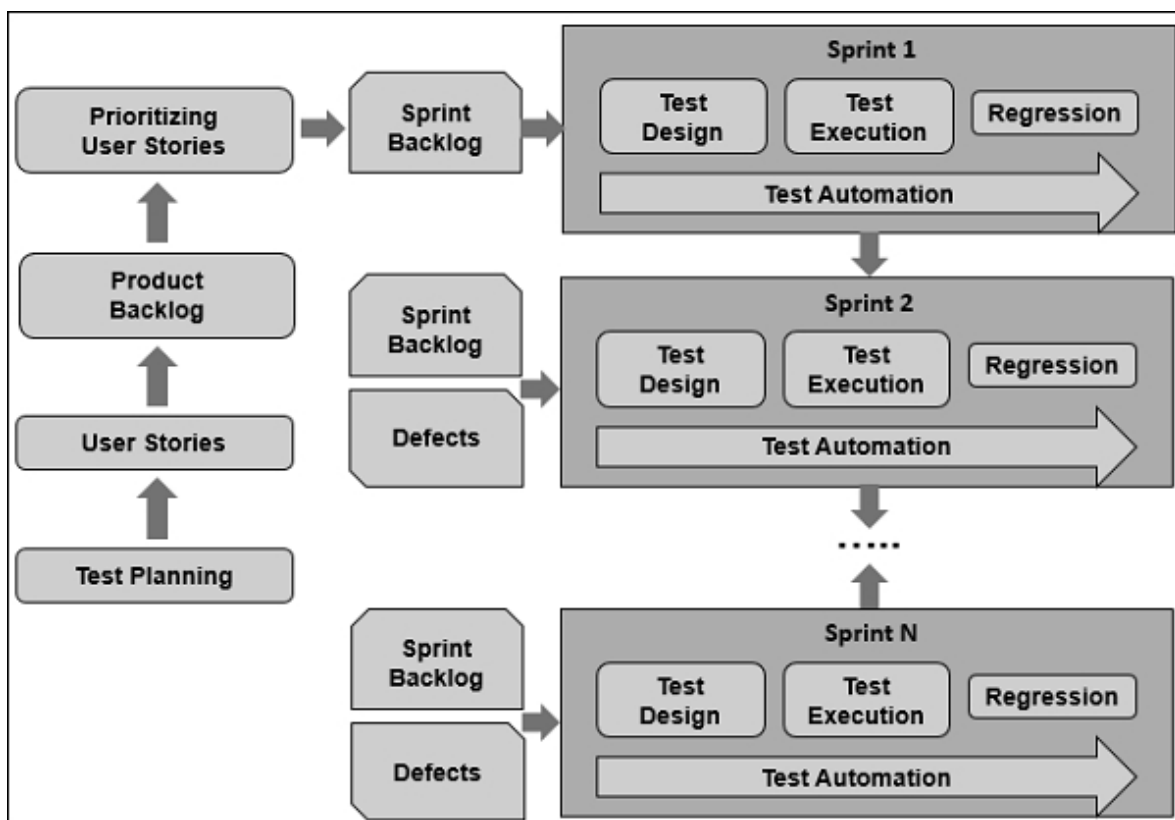
En Scrum, las actividades de prueba incluyen:

- Contribuir a las historias de usuario según el comportamiento esperado del sistema representado como casos de prueba
- Planificación de lanzamientos basada en el esfuerzo de prueba y los defectos
-

Planificación de Sprint basada en historias de usuarios y defectos

- Ejecución de Sprint con pruebas continuas
- Prueba de regresión después de completar Sprint
- Informar los resultados de la prueba
- Pruebas de automatización

Las pruebas son iterativas y se basan en sprints como se muestra en el diagrama que se muestra a continuación:



Pruebas ágiles: tester en equipo

Agile Development se centra en el equipo y los desarrolladores y evaluadores participan en todas las actividades de desarrollo y

proyectos. El trabajo en equipo maximiza el éxito de las pruebas en proyectos ágiles.

Un equipo Tester in Agile tiene que participar y contribuir a todas las actividades del proyecto y, al mismo tiempo, debe aprovechar la experiencia en testing.

Un evaluador ágil debe tener habilidades de prueba tradicionales. Además, el probador Agile necesita:

- Buenas habilidades interpersonales.
- Capacidad para actuar de manera positiva y orientada a soluciones con los miembros del equipo y las partes interesadas.
- Capacidad para mostrar un pensamiento crítico, escéptico y orientado a la calidad sobre el producto.
- Capacidad para ser proactivo para adquirir activamente información de los grupos de interés.
- Habilidades para trabajar eficazmente con los clientes y las partes interesadas en la definición de Historias de usuarios comprobables, los Criterios de aceptación.
- Talento para ser un buen miembro del equipo que trabaja con desarrolladores en la producción de código de calidad.
- La usabilidad de las habilidades de prueba para tener los casos de prueba correctos en el momento correcto y en el nivel correcto y ejecutarlos bien dentro de la duración del sprint.
-

Capacidad para evaluar e informar los resultados de las pruebas, el progreso de las pruebas y la calidad del producto.

-

Apertura para responder a los cambios rápidamente, incluido el cambio, la adición o la mejora de casos de prueba.

-

Potencial de autoorganización del trabajo.

-

Entusiasmo por el continuo desarrollo de habilidades.

-

Competencia en automatización de pruebas, desarrollo impulsado por pruebas (TDD), desarrollo impulsado por pruebas de aceptación (ATDD), desarrollo impulsado por el comportamiento (BDD) y pruebas basadas en la experiencia.

Papel del probador en el equipo ágil

Tester in Agile Team participa en todas las actividades del proyecto y desarrollo para aportar lo mejor de la experiencia en pruebas.

Las actividades de Agile Tester incluyen:

-

Garantizar el uso adecuado de las herramientas de prueba.

-

Configurar, usar y administrar los entornos de prueba y los datos de prueba.

-

Asesorar a otros miembros del equipo en aspectos relevantes de las pruebas.

-

Asegurarse de que se programen las tareas de prueba adecuadas durante el lanzamiento y la planificación del sprint.

- Comprender, implementar y actualizar la estrategia de prueba.
- Colaborar con los desarrolladores, los clientes y las partes interesadas para aclarar los requisitos, en términos de capacidad de prueba, coherencia e integridad.
- Realización de las pruebas adecuadas en el momento adecuado y en los niveles de prueba adecuados.
- Informar los defectos y trabajar con el equipo para resolverlos.
- Medir e informar la cobertura de la prueba en todas las dimensiones de cobertura aplicables.
- Participar en retrospectivas de sprints, sugiriendo e implementando mejoras de manera proactiva.

En el ciclo de vida ágil, un evaluador juega un papel importante en:

- Trabajo en equipo
- Planificación de pruebas
- Sprint cero
- Integración
- Prácticas de prueba ágiles

Trabajo en equipo

En Agile Development, el trabajo en equipo es fundamental y, por lo tanto, requiere lo siguiente:

-

Enfoque colaborativo : trabajar con miembros del equipo multifuncional en estrategia de prueba, planificación de prueba, especificación de prueba, ejecución de prueba, evaluación de prueba e informes de resultados de prueba. Contribuir con la experiencia en pruebas junto con otras actividades del equipo.

-

Autoorganización : planificación y organización bien dentro de los sprints para lograr los objetivos de las pruebas combinando la experiencia de otros miembros del equipo también.

-

Empoderamiento : tomar decisiones técnicas adecuadas para lograr los objetivos del equipo.

-

Compromiso - Comprometerse a comprender y evaluar el comportamiento y las características del producto según lo requieran los clientes y las partes interesadas.

-

Transparente : abierto, comunicativo y responsable.

-

Credibilidad : garantizar la credibilidad de la estrategia de prueba, su implementación y ejecución. Mantener informados a los clientes y las partes interesadas sobre la estrategia de prueba.

-

Abierto a comentarios : participar en retrospectivas de sprint para aprender tanto de los éxitos como de los fracasos. Buscar comentarios de los clientes y actuar de manera rápida y adecuada para garantizar entregas de calidad.

-

Resiliente : responder a los cambios.

Planificación de pruebas

La planificación de la prueba debe comenzar durante la planificación del lanzamiento y actualizarse durante cada sprint. La planificación de la prueba debe cubrir las siguientes tareas:

- Definición del alcance de la prueba, el alcance de la prueba, los objetivos de prueba y sprint.
- Decidir sobre el entorno de prueba, las herramientas de prueba, los datos de prueba y las configuraciones.
- Asignación de pruebas de funciones y características.
- Programar tareas de prueba y definir la frecuencia de las pruebas.
- Identificación de métodos de prueba, técnicas, herramientas y datos de prueba.
- Determinar los requisitos previos, como las tareas anteriores, la experiencia y la formación.
- Identificar dependencias como funciones, código, componentes del sistema, proveedor, tecnología, herramientas, actividades, tareas, equipos, tipos de prueba, niveles de prueba y limitaciones.
- Establecer prioridades considerando la importancia y las dependencias del cliente / usuario.
- Llegando al tiempo de duración y esfuerzo necesarios para realizar la prueba.
-

Identificación de tareas en la planificación de cada sprint.

Sprint cero

Sprint Zero implica actividades de preparación antes del primer sprint. Un evaluador debe colaborar con el equipo en las siguientes actividades:

- Alcance de identificación
- División de historias de usuarios en sprints
- Creando la arquitectura del sistema
- Planificación, adquisición e instalación de herramientas (incluidas herramientas de prueba)
- Creación de la estrategia de prueba inicial para todos los niveles de prueba
- Definición de métricas de prueba
- Especificar los criterios de aceptación, también denominada definición de "Listo"
- Definición de criterios de salida
- Creando tablero Scrum
- Establecer la dirección de las pruebas a lo largo de los sprints

Integración

En Agile, un producto funcional de calidad debe estar listo para su lanzamiento en cualquier momento del ciclo de vida del desarrollo. Esto implica una integración continua como parte del desarrollo. Un evaluador ágil debe admitir la integración continua con pruebas continuas.

Para lograr esto, un evaluador debe:

- Comprender la estrategia de integración.
- Identifique todas las dependencias entre funciones y características.

Prácticas de prueba ágiles

Un evaluador ágil necesita adaptar las prácticas ágiles para realizar pruebas en un proyecto ágil.

- **Emparejamiento** : dos miembros del equipo trabajan juntos en el mismo teclado. Como uno de ellos prueba, el otro revisa / analiza las pruebas. Los dos miembros del equipo pueden
 - Un probador y un desarrollador
 - Un probador y un analista de negocios
 - Dos probadores
- **Diseño de prueba incremental** : los casos de prueba se crean a partir de historias de usuarios, comenzando con pruebas simples y pasando a pruebas más complejas.
- **Mapas mentales** : un mapa mental es un diagrama para organizar la información visualmente. Los mapas mentales se pueden utilizar como una herramienta eficaz en las pruebas ágiles, mediante la cual se puede organizar la información sobre las sesiones de prueba necesarias, las estrategias de prueba y los datos de prueba.

Pruebas ágiles: seguimiento de actividades

El estado de la prueba se puede comunicar:

- Durante las reuniones diarias de pie
- Usar herramientas de gestión de pruebas estándar
- A través de mensajeros

El estado de la prueba determinado por el estado de aprobación de la prueba es crucial para decidir si la tarea está "Terminada". Hecho

significa que todas las pruebas para la tarea pasan.

Progreso de la prueba

El progreso de la prueba se puede rastrear usando -

- Tableros de Scrum (Tableros de tareas ágiles)
- Gráficos de quema
- Resultados de pruebas automatizadas

Test Progress también tiene un impacto directo en el progreso del desarrollo. Esto se debe a que una historia de usuario se puede mover al estado **Listo** solo después de que se alcancen los Criterios de aceptación. Esto, a su vez, lo decide el estado de la prueba, ya que los criterios de aceptación se juzgan mediante un estado de la prueba.

Si hay retrasos o bloqueos en el progreso de la prueba, todo el equipo discute y trabaja en colaboración para resolverlos.

En Agile Projects, los cambios ocurren con bastante frecuencia. Cuando se producen muchos cambios, podemos esperar que el estado de la prueba, el progreso de la prueba y la calidad del producto evolucionen constantemente. Los probadores ágiles necesitan llevar esa información al equipo para que se puedan tomar las decisiones adecuadas en el momento adecuado para mantenerse en el camino correcto para completar con éxito cada iteración.

Cuando ocurren cambios, pueden afectar las características existentes de iteraciones anteriores. En tales casos, las pruebas manuales y automatizadas deben actualizarse para hacer frente de manera eficaz al riesgo de regresión. También se necesitan pruebas de regresión.

Calidad del producto

Las métricas de calidad del producto incluyen:

- Pruebas pasa / no pasa
- Defectos encontrados / solucionados
- Cobertura de prueba

- Tasas de aprobación / reprobación de la prueba
- Tasas de detección de defectos
- Densidad de defectos

Automatizar la recopilación y generación de informes de métricas de calidad del producto ayuda a:

- Mantener la transparencia.
- Recopilación de todas las métricas relevantes y necesarias en el momento adecuado.
- Informes inmediatos sin retrasos en la comunicación.
- Permitir que los evaluadores se concentren en las pruebas.
- Filtrar el mal uso de métricas.

Para asegurar la calidad general del producto, el equipo Agile necesita obtener comentarios del cliente sobre si el producto cumple con las expectativas del cliente. Esto debe llevarse a cabo al final de cada iteración, y la retroalimentación será una entrada para las iteraciones posteriores.

Factores claves del éxito

En proyectos ágiles, se pueden entregar productos de calidad si las pruebas ágiles tienen éxito.

Los siguientes puntos deben tenerse en cuenta para el éxito de las pruebas ágiles:

- Las pruebas ágiles se basan en enfoques de prueba primero y continuo. Por lo tanto, las herramientas de prueba tradicionales, que se basan en el enfoque de prueba final, pueden no ser adecuadas. Por lo tanto, al elegir las herramientas de prueba en proyectos ágiles, se debe verificar la alineación con las pruebas ágiles.
- Reduzca el tiempo total de prueba automatizando las pruebas al principio del ciclo de vida del desarrollo.
-

Los evaluadores ágiles deben mantener su ritmo para alinearse con el calendario de lanzamiento de desarrollo. Por lo tanto, la planificación, el seguimiento y la nueva planificación adecuados de las actividades de prueba deben realizarse sobre la marcha con la calidad del producto como objetivo.

-

Las pruebas manuales representan el 80% de las pruebas en los proyectos. Por lo tanto, los probadores con experiencia deben ser parte del equipo Agile.

-

La participación de estos probadores con experiencia durante todo el ciclo de vida del desarrollo hace que todo el equipo se concentre en un producto de calidad que cumpla con las expectativas del cliente.

-

Definir historias de usuarios enfatizando el comportamiento del producto esperado por los usuarios finales.

-

Identificar los criterios de aceptación a nivel de historia de usuario / nivel de tarea según las expectativas del cliente.

-

Estimación del esfuerzo y la duración de las actividades de prueba.

-

Planificación de actividades de prueba.

-

Alinearse con el equipo de desarrollo para garantizar la producción de código que cumpla con los requisitos con un diseño de prueba inicial.

-

Prueba primero y prueba continua para garantizar que se alcance el estado de finalizado que cumpla con los criterios de aceptación en el momento esperado.

-

Asegurar las pruebas en todos los niveles dentro del sprint.

-

Prueba de regresión al final de cada sprint.

-

Recopilar y analizar métricas de productos que sean útiles para el éxito del proyecto.

-

Analizar los defectos para identificar cuáles deben corregirse en el Sprint actual y cuáles pueden retrasarse en Sprints posteriores.

-

Centrándose en lo importante desde el punto de vista del Cliente.

Lisa Crispin ha definido siete factores clave para el éxito de las pruebas ágiles:

-

Enfoque de equipo completo : en este tipo de enfoque, los desarrolladores capacitan a los evaluadores y los evaluadores capacitan a otros miembros del equipo. Esto ayuda a todos a comprender todas las tareas del proyecto, por lo que la colaboración y la contribución tendrán el máximo beneficio. La colaboración de los probadores con los clientes también es un factor importante para establecer sus expectativas desde el principio y traducir los criterios de aceptación a los necesarios para pasar la prueba.

-

Mentalidad de prueba ágil : los evaluadores son proactivos para mejorar continuamente la calidad y colaborar constantemente con el resto del equipo.

•

Automaticice las pruebas de regresión : diseño para la capacidad de **prueba** e impulse el desarrollo con pruebas. Empiece de forma sencilla y permita que el equipo elija las herramientas. Esté preparado para brindar consejos.

•

Proporcione y obtenga comentarios : dado que este es un valor ágil central, todo el equipo debe estar abierto a recibir comentarios. Como los evaluadores son proveedores de comentarios expertos, es necesario centrarse en la información necesaria y relevante. A cambio, al obtener comentarios, debe adaptarse a los cambios y las pruebas de los casos de prueba.

•

Construya una base de prácticas ágiles centrales : céntrese en las pruebas junto con la codificación, la integración continua, los entornos de prueba colaborativos, el trabajo incremental, la aceptación de los cambios y el mantenimiento de la sinergia.

•

Colaborar con los clientes : obtenga ejemplos, comprenda y verifique el mapeo de requisitos para el comportamiento del producto, establezca los criterios de aceptación y obtenga comentarios.

•

Mire el panorama general : impulse el desarrollo con pruebas y ejemplos orientados a la empresa utilizando datos de prueba del mundo real y pensando en los impactos en otras áreas.

Pruebas ágiles: atributos importantes

En este capítulo, veremos algunos atributos importantes de Agile Testing.

Beneficios de las pruebas ágiles

Los beneficios de las pruebas ágiles son:

- Satisfacción del cliente mediante un producto rápido, continuo y completamente probado y la búsqueda de comentarios del cliente.
- Los clientes, desarrolladores y evaluadores interactúan continuamente entre sí, reduciendo así el tiempo de ciclo.
- Los probadores ágiles participan en la definición de requisitos y contribuyen con su experiencia en pruebas para centrarse en lo que es viable.
- Los probadores ágiles participan en la estimación, evaluando el esfuerzo y el tiempo de las pruebas.
- Diseño de prueba inicial que refleja los criterios de aceptación.
- Pruebas de requisitos consolidados por todo el equipo, evitando inconvenientes.
- Enfoque constante en la calidad del producto por parte de todo el equipo.
- La definición del estado **Terminado que** refleja el paso de las pruebas garantiza que se cumpla el requisito.
- Retroalimentación continua sobre retrasos o bloqueos para que la resolución se pueda realizar de inmediato con el

esfuerzo de todo el equipo.

- Respuestas rápidas a los requisitos cambiantes y acomodarlos pronto.
- Prueba de regresión impulsada por la integración continua.
- Sin demoras entre el desarrollo y las pruebas. prueba primero, se siguen enfoques de prueba continua.
- Las pruebas de automatización se implementaron al principio del ciclo de vida del desarrollo, lo que reduce el tiempo y el esfuerzo total de las pruebas.

Mejores prácticas en pruebas ágiles

Siga las mejores prácticas que se detallan a continuación:

- Inclusión de probadores con experiencia en todo tipo de pruebas a todos los niveles.
- Testers participando en la definición de requisitos, colaborando con los clientes sobre el comportamiento esperado del producto.
- Los probadores comparten comentarios continuamente con los desarrolladores y el cliente.
- Pruebe los enfoques de prueba primero y continuo para alinearse con el trabajo de desarrollo.
- Seguimiento del estado de la prueba y el progreso de la prueba de forma rápida y constante con el objetivo de ofrecer un producto de calidad

- Pruebas de automatización al principio del ciclo de vida del desarrollo para reducir el tiempo del ciclo.
- Para realizar pruebas de regresión, aproveche las pruebas de automatización como una forma eficaz.

Desafíos en las pruebas ágiles

Existen los siguientes desafíos en las pruebas ágiles:

- La falta de comprensión del enfoque ágil y sus limitaciones por parte de la empresa y la dirección puede generar expectativas inalcanzables.
- Agile sigue el enfoque de todo el equipo, pero no todos conocen los conceptos básicos de las prácticas de prueba. Se aconseja a los evaluadores que entrenen a los demás, pero en un escenario real puede ser impracticable con Sprints encuadrados en el tiempo (iteraciones).
- Test First Approach requiere que los Desarrolladores basen la codificación en los Comentarios del Tester, pero en escenarios reales, los Desarrolladores están más acostumbrados a basar la codificación en los Requisitos provenientes del Cliente o la Empresa.
- La responsabilidad por el producto de calidad es de todo el equipo ágil, pero en las etapas iniciales los desarrolladores pueden no centrarse en la calidad, ya que están más en el modo de implementación.
- La integración continua requiere pruebas de regresión que requieren un esfuerzo considerable, incluso si deben automatizarse.

- Los probadores pueden adaptarse a los cambios con la mentalidad ágil, pero acomodar los cambios de prueba y las pruebas resultantes puede ser impracticable para terminar durante el Sprint.
- Se recomienda la automatización temprana para reducir el tiempo y el esfuerzo de prueba manual. Pero, en el escenario real, llegar a las Pruebas que se pueden automatizar y automatizarlas requiere Tiempo y Esfuerzo.

Pautas de prueba ágiles

Utilice las siguientes pautas al realizar pruebas ágiles.

- Participe en la planificación de la versión para identificar las actividades de prueba necesarias y crear la versión inicial del plan de prueba.
- Participe en la sesión de estimación para llegar al esfuerzo y la duración de la prueba de modo que las actividades de prueba se acomoden en las iteraciones.
- Participe en la definición de la historia del usuario para llegar a casos de prueba de aceptación.
- Participe en cada reunión de planificación de Sprint para comprender el alcance y actualizar el plan de prueba.
- Colabore continuamente con el equipo de desarrollo durante el Sprint para que las pruebas y la codificación sean un éxito dentro del Sprint.
- Participe en reuniones diarias de stand-up y transmita retrasos o bloqueos de prueba, si los hay, para recibir una

resolución inmediata.

-

Realice un seguimiento e informe del estado de la prueba, el progreso de la prueba y la calidad del producto con regularidad.

-

Esté preparado para adaptarse a los cambios, respondiendo con modificaciones a los casos de prueba, datos de prueba.

-

Participe en las retrospectivas de Sprint para comprender y contribuir con las mejores prácticas y lecciones aprendidas.

-

Colabore para obtener comentarios de los clientes en cada Sprint.

Pruebas ágiles: cuadrantes

Como en el caso de las pruebas tradicionales, las pruebas ágiles también deben cubrir todos los niveles de prueba.

- Examen de la unidad
- Pruebas de integración
- Prueba del sistema
- Pruebas de aceptación del usuario

Examen de la unidad

- Hecho junto con la codificación, por el desarrollador
- Apoyado por Tester que escribe casos de prueba asegurando una cobertura de diseño del 100%
- Los casos de prueba unitaria y los resultados de las pruebas unitarias deben revisarse
- Los defectos importantes no resueltos (según la prioridad y la gravedad) no se dejan
- Todas las pruebas unitarias están automatizadas

Pruebas de integración

- Se realiza junto con la integración continua a medida que avanzan los Sprints
- Hecho al final después de que se completen todos los Sprints
- Todos los requisitos funcionales se prueban
- Todas las interfaces entre unidades se prueban
- Se informan todos los defectos
- Las pruebas se automatizan siempre que sea posible

Prueba del sistema

- Hecho a medida que avanza el desarrollo
- Se prueban las historias, características y funciones de los usuarios
- Pruebas realizadas en entorno de producción
- Se realizan pruebas de calidad (rendimiento, fiabilidad, etc.)
- Se informan los defectos
- Las pruebas se automatizan siempre que sea posible

Pruebas de aceptación del usuario

- Realizado al final de cada Sprint y al final del proyecto
- Realizado por el Cliente. El equipo recibe comentarios
- La retroalimentación será una entrada para Sprints posteriores
- Las historias de usuario en un Sprint se verifican previamente para que se puedan probar y tienen criterios de aceptación definidos

Tipos de prueba

- Pruebas de componentes (pruebas unitarias)
- Pruebas funcionales (pruebas de historias de usuario)

- Pruebas no funcionales (rendimiento, carga, estrés, etc.)
- Prueba de aceptación

Las pruebas pueden ser totalmente manuales, totalmente automatizadas, una combinación de manual y automatizada o manuales compatibles con herramientas.

Programación de soporte y pruebas de productos críticos

Las pruebas pueden ser para:

- **Desarrollo de soporte (programación de soporte) :** los programadores utilizan las pruebas de programación de soporte.
 - Para decidir qué código necesitan escribir para lograr un cierto comportamiento de un sistema
 - Qué pruebas deben ejecutarse después de la codificación para garantizar que el nuevo código no obstaculice el resto de los comportamientos del sistema
- **Solo verificación (producto crítico) :** las pruebas de producto crítico se utilizan para descubrir deficiencias en el producto terminado

Pruebas de orientación empresarial y tecnológica

Para decidir qué pruebas se realizarán y cuándo, debe determinar si una prueba es:

- Orientación empresarial, o
- Frente a la tecnología

Pruebas de cara al negocio

Una prueba es una prueba orientada a los negocios si responde a las preguntas enmarcadas con palabras del dominio empresarial. Estos son entendidos por los expertos en negocios y les interesarían para que el comportamiento del sistema se pueda explicar en el escenario en tiempo real.

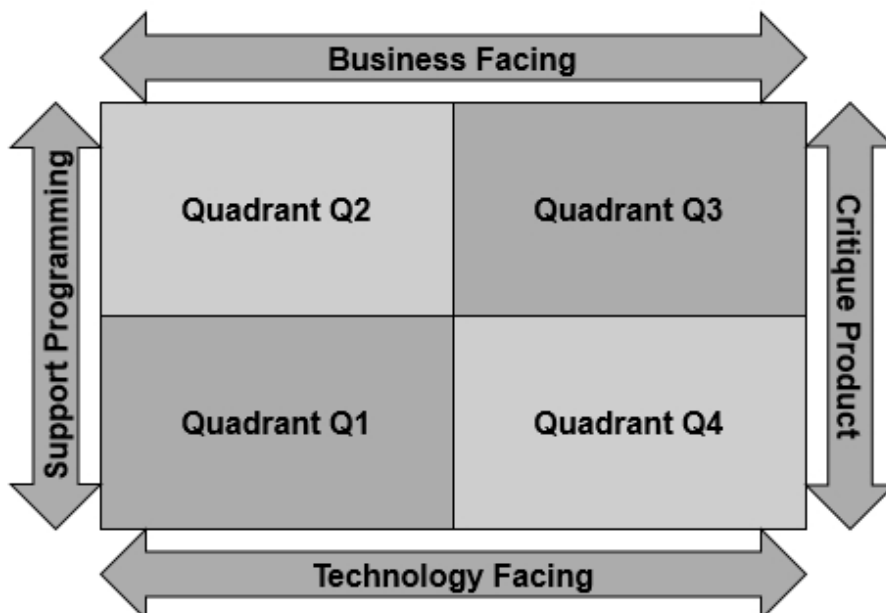
Pruebas de tecnología

Una prueba es una prueba orientada a la tecnología si responde a las preguntas enmarcadas con palabras del dominio de la tecnología. Los programadores entienden lo que debe implementarse en función de las aclaraciones sobre tecnología.

Estos dos aspectos de los tipos de prueba se pueden ver utilizando los Cuadrantes de prueba ágiles definidos por Brian Marick.

Cuadrantes de pruebas ágiles

Combinando los dos aspectos de los tipos de prueba, Brian Marick deriva los siguientes cuadrantes de prueba ágiles:



Los cuadrantes de pruebas ágiles proporcionan una taxonomía útil para ayudar a los equipos a identificar, planificar y ejecutar las pruebas necesarias.

-

Cuadrante Q1 : nivel de unidad, orientación tecnológica y apoyo a los desarrolladores. Las pruebas unitarias pertenecen a este cuadrante. Estas pruebas pueden ser pruebas automatizadas.

-

Cuadrante Q2 : nivel de sistema, orientación empresarial y comportamiento del producto conforme. Las pruebas funcionales pertenecen a este cuadrante. Estas pruebas son manuales o automatizadas.

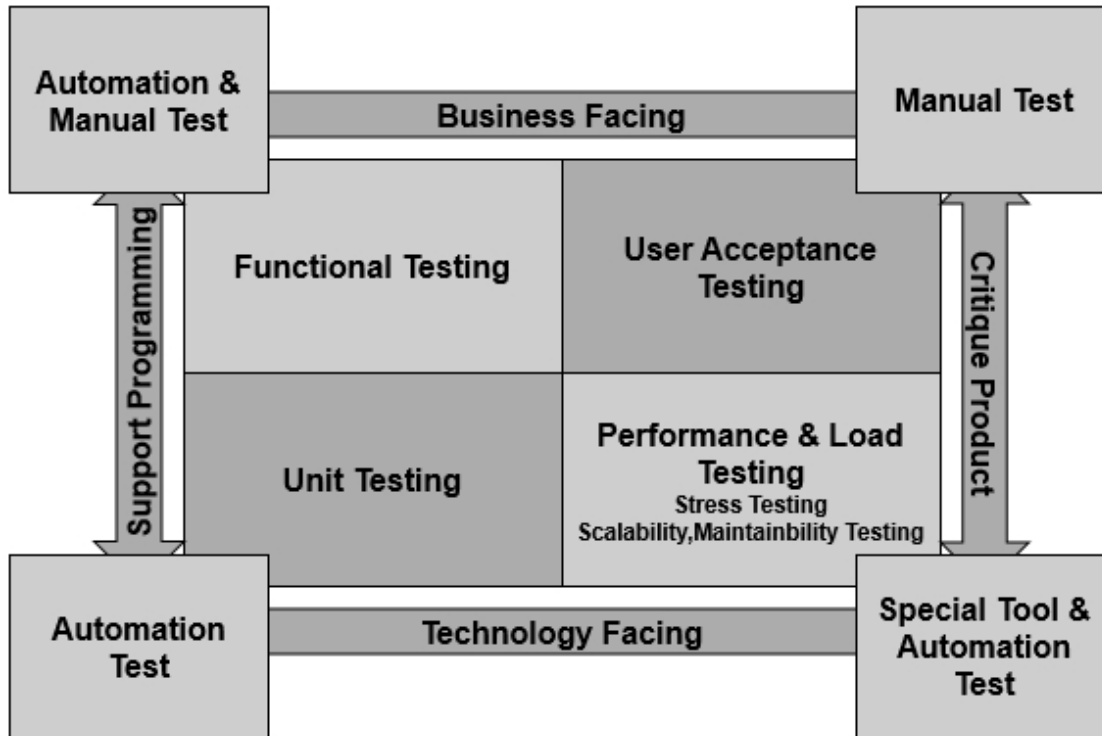
-

Cuadrante Q3 : nivel de aceptación del sistema o del usuario, orientación empresarial y enfoque en escenarios en tiempo real. Las pruebas de aceptación del usuario pertenecen a este cuadrante. Estas pruebas son manuales.

-

Cuadrante Q4 : nivel de aceptación operativa o del sistema, tecnología de cara y enfoque en pruebas de rendimiento, carga, estrés, capacidad de mantenimiento y escalabilidad. Se pueden utilizar herramientas especiales para estas pruebas junto con las pruebas de automatización.

Combinando estos, los Cuadrantes de prueba ágiles que reflejan **Qué-Prueba-Cuándo** se pueden visualizar de la siguiente manera:



Pruebas ágiles - Scrum

Scrum aboga **por el enfoque de equipo completo**, en el sentido de que cada miembro del equipo debe participar en todas las actividades del proyecto. El equipo de Scrum se autoorganiza y se responsabiliza de los entregables del proyecto. La toma de decisiones se deja en manos del equipo, lo que da como resultado que se tomen las acciones adecuadas en el momento adecuado sin retrasos. Este enfoque también fomenta el uso adecuado del talento del equipo en lugar de limitarse a una actividad. Los probadores también participan en todos los proyectos y actividades de desarrollo aportando su experiencia en pruebas.

Todo el equipo trabaja en conjunto en la estrategia de prueba, planificación de prueba, especificación de prueba, ejecución de prueba, evaluación de prueba e informes de resultados de prueba.

Creación colaborativa de historias de usuario

Los probadores participan en la creación de historias de usuario. Los probadores aportan sus ideas sobre el posible comportamiento del sistema. Esto ayuda a que el cliente y / o el usuario final comprendan el sistema en el entorno real y así obtengan claridad sobre lo que realmente quieren como resultado. Esto da como resultado una congelación más rápida de los requisitos y también reduce la probabilidad de cambios en los requisitos más adelante.

Los evaluadores también elaboran los Criterios de aceptación para cada escenario acordado por el cliente.

Los probadores contribuyen a la creación de historias de usuarios comprobables.

Planificación de lanzamiento

La planificación de la versión se realiza para todo el proyecto. Sin embargo, el marco de Scrum implica una toma de decisiones iterativa a medida que se obtiene más información a su debido tiempo de la ejecución de sprints. Por lo tanto, la sesión de Planificación de la versión al comienzo del proyecto no necesita producir un plan de entrega detallado para todo el proyecto. Puede actualizarse continuamente, a medida que se disponga de información relevante.

Cada sprint-end no necesita tener un lanzamiento. Una liberación puede ser después de un grupo de sprints. El criterio principal de un lanzamiento es entregar valor comercial al cliente. El equipo decide la duración del sprint con la planificación del lanzamiento como entrada.

La planificación de la versión es la base del enfoque de prueba y el plan de prueba para la versión. Los probadores estiman el esfuerzo de prueba y planifican las pruebas para el lanzamiento. Cuando los planes de lanzamiento cambian, los evaluadores deben manejar los cambios, obtener una base de prueba adecuada considerando un

contexto más amplio de lanzamiento. Los probadores también proporcionan el esfuerzo de prueba que se requiere al final de todos los sprints.

Planificación de Sprint

La planificación del sprint se realiza al comienzo de cada sprint. El backlog del sprint se crea con las historias de usuario recogidas del backlog del producto para su implementación en ese sprint en particular.

Los probadores deberían:

- Determinar la capacidad de prueba de las historias de usuario seleccionadas para el sprint.
- Crea pruebas de aceptación
- Definir niveles de prueba
- Identificar la automatización de pruebas

Los evaluadores actualizan el plan de pruebas con las estimaciones del esfuerzo y la duración de las pruebas en el sprint. Esto asegura la provisión de tiempo para las pruebas requeridas durante los sprints encuadrados en el tiempo y también la responsabilidad del esfuerzo de prueba.

Análisis de prueba

Cuando comienza un sprint, mientras los desarrolladores realizan el análisis de la historia para el diseño y la implementación, los evaluadores realizan análisis de prueba para las historias en el backlog del sprint. Tester crea los casos de prueba necesarios, tanto pruebas manuales como automatizadas.

Pruebas

Todos los miembros del equipo Scrum deben participar en las pruebas.

-

Los desarrolladores ejecutan las pruebas unitarias a medida que desarrollan el código para las historias de usuario. Las pruebas unitarias se crean en cada sprint, antes de que se escriba el código. Los casos de prueba unitaria se derivan de especificaciones de diseño de bajo nivel.

-

Los probadores realizan características funcionales y no funcionales de las historias de usuario.

-

Los evaluadores asesoran a los otros miembros del equipo scrum con su experiencia en pruebas para que todo el equipo tenga una responsabilidad colectiva por la calidad del producto.

-

Al final del sprint, el cliente y / o los usuarios finales llevan a cabo las pruebas de aceptación del usuario y brindan comentarios al equipo de scrum. Esto se forma como una entrada para el próximo sprint.

-

Los resultados de las pruebas se recopilan y mantienen.

Pruebas de automatización

Las pruebas de automatización tienen una gran importancia en los equipos Scrum. Los evaluadores dedican tiempo a crear, ejecutar, monitorear y mantener pruebas y resultados automatizados. Como los cambios pueden ocurrir en cualquier momento en los proyectos de scrum, los evaluadores deben adaptarse a las pruebas de las características cambiadas y también a las pruebas de regresión involucradas. Las pruebas de automatización facilitan la gestión del esfuerzo de prueba asociado con los cambios. Las pruebas automatizadas en todos los niveles facilitan lograr una integración continua. Las pruebas automatizadas se ejecutan mucho más rápido que las pruebas manuales sin ningún esfuerzo adicional.

Las pruebas manuales se centran más en las pruebas exploratorias, la vulnerabilidad del producto y la predicción de defectos.

Automatización de actividades de prueba

La automatización de las actividades de prueba reduce la carga del trabajo repetido y genera ahorros de costos. Automatizar

- Generación de datos de prueba
- Carga de datos de prueba
- Construya la implementación en el entorno de prueba
- Gestión del entorno de prueba
- Comparación de salida de datos

Pruebas de regresión

En un sprint, los probadores prueban el código nuevo / modificado en ese sprint. Sin embargo, los evaluadores también deben asegurarse de que el código desarrollado y probado en los sprints anteriores también funcione junto con el nuevo código. Por lo tanto, se le da importancia a las pruebas de regresión en scrum. Las pruebas de regresión automatizadas se ejecutan en integración continua.

Gestión de la configuración

En los proyectos Scrum se utiliza un sistema de gestión de la configuración que utiliza marcos de prueba y compilación automatizados. Esto permite ejecutar análisis estáticos y pruebas unitarias repetidamente a medida que se ingresa un nuevo código en el Sistema de gestión de la configuración. También gestiona la integración continua del nuevo código con el sistema. Las pruebas de regresión automatizadas se ejecutan durante la integración continua.

Los casos de prueba manuales, las pruebas automatizadas, los datos de prueba, los planes de prueba, la estrategia de prueba y otros

artefactos de prueba deben estar controlados por versiones y requieren los permisos de acceso pertinentes para garantizarse. Esto se puede lograr manteniendo los artefactos de prueba en el sistema de gestión de la configuración.

Prácticas de prueba ágiles

Los probadores de un equipo Scrum pueden seguir las siguientes prácticas ágiles:

- **Emparejamiento** : dos miembros del equipo se sientan juntos y trabajan en colaboración. Las dos personas pueden ser dos probadores o un probador y un desarrollador.
- **Diseño de prueba incremental** : los casos de prueba se desarrollan a medida que los Sprints progresan de manera incremental y se suman las historias de usuario.

Métricas ágiles

Durante el desarrollo del software, la recopilación y el análisis de métricas ayudan a mejorar el proceso y, por lo tanto, a lograr una mejor productividad, entregables de calidad y satisfacción del cliente. En el desarrollo basado en Scrum, esto es posible y los evaluadores deben prestar atención a las métricas que necesitan.

Se sugieren varias métricas para el desarrollo de Scrum. Las métricas importantes son:

- **Proporción de Sprints exitosos** - $(\text{Número de Sprints exitosos} / \text{Número total de Sprints}) * 100$. Un Sprint exitoso es aquel en el que el equipo podría cumplir con su compromiso.
- **Velocidad** : la **velocidad de** un equipo se basa en la cantidad de puntos de historia que un equipo ganó

durante un sprint. Los puntos de historia son la medida de las historias de usuario contadas durante la estimación.

-

Factor de enfoque - $(\text{Velocidad} / \text{Capacidad de trabajo del equipo}) / 100$. El factor de enfoque es el porcentaje del esfuerzo del equipo que da como resultado historias terminadas.

-

Exactitud de la estimación - $(\text{Esfuerzo estimado} / \text{Esfuerzo real}) / 100$. La precisión de la estimación es la capacidad del equipo para estimar el esfuerzo con precisión.

-

Sprint Burndown : trabajo (en Story Points o en horas) restante vs. Trabajo que debe quedar idealmente restante (según la estimación).

-

Si es más, significa que el equipo ha asumido más trabajo del que puede hacer.

-

Si es menor, significa que el equipo no estimó con precisión.

-

Recuento de defectos: número de defectos en un Sprint. El recuento de defectos es la cantidad de defectos en el software en comparación con la acumulación.

-

Severidad de los defectos : los defectos se pueden clasificar como menores, mayores y críticos según su gravedad. Los evaluadores pueden definir la categorización.

Retrospectivas de Sprint

En Sprint Retrospectives, todos los miembros del equipo participarán. Ellos comparten -

- Las cosas que salieron bien
- Métrica
- El alcance de las mejoras
- Elementos de acción para aplicar

Pruebas ágiles: métodos

En las pruebas ágiles, los métodos de prueba más utilizados provienen de las prácticas tradicionales y están alineados con el principio: prueba temprana. Los casos de prueba se escriben antes de escribir el código. El énfasis está en la prevención, detección y eliminación de defectos ejecutando los tipos de prueba correctos en el momento correcto y al nivel correcto.

En este capítulo, comprenderá los métodos:

- Desarrollo basado en pruebas (TDD)
- Desarrollo impulsado por pruebas de aceptación (ATDD)
- Desarrollo impulsado por el comportamiento (BDD)

Desarrollo basado en pruebas

En el método Test Driven Development (TDD), el código se desarrolla en base al enfoque Testfirst dirigido por Automated Test Cases. Un caso de prueba se escribe primero en fallar, el código se desarrolla en base a eso para garantizar que la prueba pase. El método se repite, la refactorización se realiza mediante el desarrollo de código.

TDD se puede entender con la ayuda de los siguientes pasos:

- **Paso 1** : escriba un caso de prueba para reflejar el comportamiento esperado de la funcionalidad del código que debe escribirse.
-

Paso 2 : ejecuta la prueba. La prueba falla porque el código aún no está desarrollado.

-

Paso 3 : desarrolle código basado en el caso de prueba.

-

Paso 4 : vuelva a ejecutar la prueba. Esta vez, la prueba debe pasar mientras se codifica la funcionalidad. Repita el Paso (3) y el Paso (4) hasta que pase la prueba.

-

Paso 5 : Refactorice el código.

-

Paso 6 : vuelva a ejecutar la prueba para asegurarse de que pase.

Repita el **Paso 1 al Paso 6** agregando casos de prueba para agregar funcionalidad. Las pruebas agregadas y las pruebas anteriores se ejecutan cada vez para garantizar que el código se esté ejecutando como se esperaba. Para agilizar este proceso, las pruebas están automatizadas.

Las pruebas pueden ser a nivel de unidad, integración o sistema. Debe garantizarse la comunicación constante entre probadores y desarrolladores.

Desarrollo impulsado por pruebas de aceptación

En el método Acceptance Test Driven Development (ATDD), el código se desarrolla en base al enfoque de prueba primero dirigido por Acceptance Test Cases. La atención se centra en los criterios de aceptación y los casos de prueba de aceptación escritos por los probadores durante la creación de historias de usuario en colaboración con el cliente, los usuarios finales y las partes interesadas relevantes.

-

Paso 1 : redactar casos de prueba de aceptación junto con historias de usuarios en colaboración con el cliente y los usuarios.

•

Paso 2 : defina los criterios de aceptación asociados.

•

Paso 3 : desarrolle un código basado en las pruebas de aceptación y los criterios de aceptación.

•

Paso 4 : ejecute las pruebas de aceptación para asegurarse de que el código se esté ejecutando como se esperaba.

•

Paso 5 : automatice las pruebas de aceptación. Repita el **Paso 3 - Paso 5** hasta que se implementen todas las historias de usuario en la iteración.

•

Paso 6 : automatice las pruebas de regresión.

•

Paso 7 : ejecute las pruebas de regresión automatizadas para garantizar la regresión continua.

Desarrollo impulsado por el comportamiento (BDD)

El desarrollo impulsado por comportamiento (BDD) es similar al desarrollo impulsado por pruebas (TDD), y el enfoque está en probar el código para garantizar el comportamiento esperado del sistema.

En BDD, se usa un lenguaje como el inglés para que tenga sentido para los usuarios, evaluadores y desarrolladores. Asegura -

- Comunicación continua entre los usuarios, testers y desarrolladores.
- Transparencia sobre lo que se está desarrollando y probando.

Pruebas ágiles: técnicas

Las técnicas de prueba de las pruebas tradicionales también se pueden utilizar en las pruebas ágiles. Además de estos, en los proyectos Agile se utilizan técnicas y terminologías de prueba específicas de Agile.

Base de prueba

En proyectos ágiles, la cartera de productos reemplaza los documentos de especificación de requisitos. El contenido de la acumulación de productos suele ser historias de usuarios. Los requisitos no funcionales también se tienen en cuenta en las historias de usuario. Por lo tanto, la base de prueba en los proyectos ágiles es la historia del usuario.

Para garantizar las pruebas de calidad, lo siguiente también se puede considerar adicionalmente como base de prueba:

- Experiencia de iteraciones anteriores del mismo proyecto o proyectos anteriores.
- Funciones existentes, arquitectura, diseño, código y características de calidad del sistema.
- Datos de defectos de los proyectos actuales y pasados.
- Comentarios de los clientes.
- Documentación del usuario.

Definición de Terminado

La Definición de Terminado (DoD) es el criterio que se utiliza en los proyectos ágiles para garantizar la finalización de una actividad en la cartera de Sprint. El DoD puede variar de un equipo Scrum a otro, pero debe ser consistente dentro de un equipo.

DoD es una lista de verificación de actividades necesarias que garantizan la implementación de funciones y características en una historia de usuario junto con los requisitos no funcionales que forman parte de la historia de usuario. Una historia de usuario llega a la etapa Listo después de que se completan todos los elementos

de la lista de verificación del Departamento de Defensa. Un DoD se comparte en todo el equipo.

Un DoD típico para una historia de usuario puede contener:

- Criterios de aceptación comprobables detallados
- Criterios para garantizar la coherencia de la historia de usuario con las demás en la iteración
- Criterios específicos relacionados con el producto
- Aspectos de comportamiento funcional
- Características no funcionales
- Interfaces
- Requisitos de datos de prueba
- Cobertura de prueba
- Refactorización
- Requisitos de revisión y aprobación

Además del DoD para historias de usuarios, también se requiere DoD:

- en cada nivel de prueba
- para cada característica
- para cada iteración
- para liberación

Información de prueba

Un probador debe tener la siguiente información de prueba:

- Historias de usuarios que deben probarse
- Criterios de aceptación asociados
- Interfaces del sistema
- Entorno donde se espera que funcione el sistema
- Disponibilidad de herramientas
- Cobertura de prueba
- DoD

En los proyectos ágiles, dado que las pruebas no son una actividad secuencial y se supone que los probadores deben trabajar en modo colaborativo, es responsabilidad del probador:

- Obtenga la información de prueba necesaria de forma continua.
- Identifique las lagunas de información que afectan las pruebas.
- Resuelva las brechas en colaboración con otros miembros del equipo.
- Decida cuándo se alcanza un nivel de prueba.
- Asegúrese de que se ejecuten las pruebas adecuadas en los momentos pertinentes.

Diseño de prueba funcional y no funcional

En proyectos ágiles, se pueden utilizar las técnicas de prueba tradicionales, pero la atención se centra en las pruebas tempranas. Los casos de prueba deben estar en su lugar antes de que comience la implementación.

Para el diseño de prueba funcional, los probadores y desarrolladores pueden utilizar las técnicas tradicionales de diseño de prueba Black Box, como:

- Partición de equivalencia
- Análisis de valor límite
- Tablas de decisiones
- Transición de estado
- Árbol de clases

Para el diseño de prueba no funcional, dado que los requisitos no funcionales también son parte de cada historia de usuario, las técnicas de diseño de prueba de caja negra solo se pueden utilizar para diseñar los casos de prueba relevantes.

Prueba exploratoria

En proyectos ágiles, el tiempo es a menudo el factor de limitación para el análisis de pruebas y el diseño de pruebas. En tales casos,

las técnicas de prueba exploratoria se pueden combinar con las técnicas de prueba tradicionales.

Las pruebas exploratorias (ET) se definen como aprendizaje simultáneo, diseño de pruebas y ejecución de pruebas. En las pruebas exploratorias, el evaluador controla activamente el diseño de las pruebas a medida que se realizan y utiliza la información obtenida durante las pruebas para diseñar nuevas y mejores pruebas.

Las pruebas exploratorias son útiles para adaptarse a los cambios en los proyectos ágiles.

Pruebas basadas en riesgos

Las pruebas basadas en riesgos son pruebas basadas en el riesgo de falla y mitigan los riesgos utilizando técnicas de diseño de pruebas.

Un riesgo de calidad del producto se puede definir como un problema potencial con la calidad del producto. Los riesgos de calidad del producto incluyen:

- Riesgos funcionales
- Riesgos de desempeño no funcionales
- Riesgos de usabilidad no funcionales

Se debe realizar un análisis de riesgo para evaluar la probabilidad (probabilidad) y el impacto de cada riesgo. Luego, se priorizan los riesgos:

- Los riesgos elevados requieren pruebas exhaustivas
- Los riesgos bajos solo requieren pruebas de curso

Las pruebas se diseñan utilizando las técnicas de prueba adecuadas en función del nivel de riesgo y la característica de riesgo de cada riesgo. Luego se ejecutan pruebas para mitigar los riesgos.

Pruebas de ajuste

Las pruebas de ajuste son pruebas de aceptación automatizadas. Las herramientas Fit y FitNesse se pueden utilizar

para automatizar las pruebas de aceptación.

FIT usa JUnit, pero extiende la funcionalidad de prueba. Las tablas HTML se utilizan para mostrar los casos de prueba. Fixture es una clase Java detrás de la tabla HTML. El dispositivo toma el contenido de la tabla HTML y ejecuta los casos de prueba en el proyecto que se está probando.

Pruebas ágiles: productos de trabajo

El plan de prueba se prepara en el momento de la planificación del lanzamiento y se revisa en cada planificación de Sprint. El plan de prueba actúa como una guía para el proceso de prueba con el fin de tener la cobertura de prueba completa.

Los contenidos típicos de un plan de prueba son:

- Estrategia de prueba
- Entorno de prueba
- Cobertura de prueba
- Alcance de la prueba
- Probar el esfuerzo y el calendario
- Herramientas de prueba

En Agile Projects, todos los miembros del equipo son responsables de la calidad del producto. Por lo tanto, todos participan también en la planificación de la prueba.

La responsabilidad de los probadores es proporcionar la dirección necesaria y guiar al resto del equipo con su experiencia en pruebas.

Historias de usuarios

Las historias de usuario no están probando productos de trabajo en principio. Sin embargo, en Agile Projects, los probadores participan en la creación de historias de usuario. Los probadores escriben historias de usuarios que aportan valor al cliente y cubren diferentes comportamientos posibles del sistema.

Los evaluadores también se aseguran de que todas las Historias de usuarios sean probables y garantizan los Criterios de aceptación.

Pruebas manuales y automatizadas

Durante la primera ejecución de pruebas, se utilizan pruebas manuales. Incluyen:

- Pruebas unitarias
- Pruebas de integración
- Pruebas funcionales
- Pruebas no funcionales
- Prueba de aceptación

Luego, las pruebas se automatizan para ejecuciones posteriores.

En **el desarrollo basado en pruebas**, las pruebas unitarias se escriben primero en fallar, el código se desarrolla y prueba para garantizar que las pruebas pasen.

En **el desarrollo impulsado por** pruebas de aceptación, las pruebas de aceptación se escriben primero en fallar, el código se desarrolla y prueba para garantizar que las pruebas pasen.

En otros métodos de desarrollo, los probadores colaboran con el resto del equipo para garantizar la cobertura de la prueba.

En todos los tipos de métodos, se lleva a cabo la integración continua, que incluye pruebas de integración continua.

El equipo puede decidir cuándo y qué pruebas se automatizarán. Incluso si la automatización de las pruebas requiere esfuerzo y tiempo, las pruebas automatizadas resultantes reducen significativamente el esfuerzo y el tiempo de prueba repetitivos durante las iteraciones del Proyecto Agile. Esto, a su vez, facilita que el equipo preste más atención a las otras actividades requeridas, como nuevas historias de usuario, cambios, etc.

En **Scrum**, las iteraciones están encuadradas en el tiempo. Por lo tanto, si una prueba de historia de usuario no se puede completar en un Sprint en particular, el evaluador puede informar en la reunión diaria que la historia del usuario no puede alcanzar el estado de finalizado dentro de ese Sprint y, por lo tanto, debe mantenerse pendiente del próximo Sprint.

Resultados de la prueba

Como la mayoría de las pruebas en proyectos ágiles están automatizadas, las herramientas generan los registros de resultados de pruebas necesarios. Los evaluadores revisan los registros de resultados de las pruebas. Los resultados de la prueba deben mantenerse para cada sprint / lanzamiento.

También se puede preparar un resumen de la prueba que contenga:

- Alcance de prueba (lo que se probó y lo que no se probó)
- Análisis de defectos junto con análisis de causa raíz si es posible
- Estado de la prueba de regresión después de la corrección de defectos
- Incidencias y resolución correspondiente
- Problemas pendientes, si los hay
- Cualquier modificación requerida en la estrategia de prueba
- Métricas de prueba

Informes de métricas de prueba

En proyectos ágiles, las métricas de prueba incluyen lo siguiente para cada Sprint:

- Esfuerzo de prueba
- Exactitud de la estimación de la prueba
- Cobertura de prueba
- Cobertura de prueba automatizada
- No. de defectos
- Tasa de defectos (número de defectos por punto de historia de usuario)
- Severidad del defecto
- Es hora de corregir un defecto en el mismo Sprint (Cuesta 24 veces más reparar un error que se escapa del Sprint actual)
- No de defectos solucionados en el mismo Sprint
- Finalización de las pruebas de aceptación por parte del cliente dentro del Sprint

Revisión de Sprint e informes retrospectivos

Los evaluadores también contribuyen a la revisión de Sprint y los informes retrospectivos. Los contenidos típicos son:

- Métricas de prueba
- Los registros de resultados de la prueba revisan los resultados
- Qué salió bien y qué se puede mejorar desde Testing Point of View
- Mejores prácticas
- Lecciones aprendidas
- Cuestiones
- Comentarios de los clientes

Pruebas ágiles - Kanban

Las actividades de pruebas ágiles se pueden gestionar de forma eficaz utilizando conceptos Kanban. Lo siguiente asegura que las pruebas se completen a tiempo dentro de una iteración / sprint y, por lo tanto, se centran en la entrega de un producto de calidad.

- Las historias de usuario que se pueden probar y dimensionar de manera efectiva dan como resultado el desarrollo y las pruebas dentro de los límites de tiempo especificados.
- El límite de WIP (Work-In-Progress) permite concentrarse en un número limitado de historias de usuarios a la vez.
- El tablero Kanban que representa el flujo de trabajo visualmente ayuda a rastrear las actividades de prueba y los cuellos de botella, si los hay.
- El concepto de colaboración en equipo Kanban permite la resolución de cuellos de botella a medida que se identifican, sin tiempo de espera.

- La preparación de casos de prueba por adelantado, el mantenimiento del conjunto de pruebas a medida que avanza el desarrollo y la obtención de comentarios del cliente ayudan a eliminar los defectos dentro de la iteración / sprint.
- Se dice que la Definición de Terminado (DoD) está Terminado-Terminado en el sentido de que una Historia alcanza un estado de finalización solo después de que la prueba también se completa.

Actividades de prueba en el desarrollo de productos

En el desarrollo de productos, los lanzamientos se pueden rastrear con el tablero Kanban de funciones. Las funciones para una versión en particular se asignan al tablero Feature Kanban que rastrea visualmente el estado de desarrollo de la función.

Las características de una versión se dividen en historias y se desarrollan dentro de la versión utilizando un enfoque ágil.

Las siguientes actividades de pruebas ágiles garantizan una entrega de calidad en cada versión y también al final de todas las versiones:

- Los probadores participan en la creación de historias de usuario y, por lo tanto, se aseguran:
 - Todos los posibles Comportamientos del Sistema se capturan mediante Historias de Usuario y los Requisitos No Funcionales que forman parte de las Historias de Usuario.
 - Las historias de usuario se pueden probar.
 -

El tamaño de las historias de usuario permite que el desarrollo y las pruebas se completen (DoneDone) dentro de la iteración.

-

Tablero Kanban de Tarea Visual -

-

Muestra el estado y el progreso de las tareas.

-

Los cuellos de botella se identifican inmediatamente a medida que ocurren

-

Facilita la medición del tiempo de ciclo que luego se puede optimizar

-

La colaboración en equipo ayuda a:

-

Responsabilidad de todo el equipo por el producto de calidad

-

Resolución de cuellos de botella a medida que ocurren, ahorrando tiempo de espera

-

Contribución de cada experiencia en todas las actividades

-

Integración continua que se centra en las pruebas de integración continua

-

Automatización de pruebas para ahorrar tiempo y esfuerzo en las pruebas

-

Prevención de defectos con casos de prueba escritos anteriormente para Desarrollo y asesorar a los

Desarrolladores sobre lo que anticipan los diferentes comportamientos del Sistema:

- Límite de WIP para centrarse en un número limitado de historias de usuario a la vez
- Pruebas continuas a medida que avanza el desarrollo, para garantizar la corrección de defectos dentro de la iteración:
 - Asegure la cobertura de la prueba
 - Mantenga bajo el recuento de defectos abiertos

Exploración de historias

Story Exploration es la comunicación dentro de un equipo ágil para explorar la comprensión de la historia cuando el propietario del producto pasa una historia para su aceptación para el desarrollo.

Al propietario del producto se le ocurre la historia basándose en la funcionalidad esperada por el sistema. Los desarrolladores exploran más en cada historia antes de marcarla lista para su aceptación. Los evaluadores también participan en la comunicación desde la perspectiva de la prueba para que sea lo más comprobable posible.

La finalización de la historia se basa en una comunicación constante y continua entre el propietario del producto, los desarrolladores y los probadores.

Estimacion

La estimación ocurre en la planificación de versiones y en cada planificación de iteraciones.

En Release Planning, los probadores proporcionan:

- Información sobre qué actividades de prueba se requieren
- Estimación del esfuerzo para el mismo

En la planificación de iteraciones, los evaluadores contribuyen a decidir qué y cuántas historias se pueden incluir en una iteración. La decisión depende del esfuerzo de prueba y la estimación del programa de prueba. La estimación de la historia también refleja la estimación de la prueba.

En Kanban, Done-Done solo se logra cuando se desarrolla y se prueba una historia y se marca como completa sin defectos.

Por lo tanto, la estimación de prueba juega un papel importante en la estimación de historias.

Planificación de la historia

La planificación de la historia comienza después de que una historia ha sido estimada y asignada a la iteración actual.

Story Planning incluye las siguientes tareas de prueba:

- Preparar datos de prueba
- Ampliar las pruebas de aceptación
- Ejecutar pruebas manuales
- Realizar sesiones de pruebas exploratorias
- Automatice las pruebas de integración continua

Además de estas tareas de prueba, es posible que se requieran otras tareas, como:

- Pruebas de rendimiento
- Pruebas de regresión
- Actualizaciones de las pruebas de integración continua relacionadas

Progresión de la historia

Story Progression descubre pruebas adicionales que se requieren como resultado de la comunicación continua entre los desarrolladores y probadores. En situaciones en las que los desarrolladores necesitan más claridad sobre la implementación, los evaluadores realizan pruebas exploratorias.

La prueba continua se realiza durante la progresión de la historia e incluye la prueba de integración continua. Todo el equipo participa en las actividades de prueba.

Aceptación de la historia

La aceptación de la historia ocurre cuando la historia alcanza el estado Hecho-Hecho. es decir, la historia se desarrolla y se prueba y se señala como completa.

Se dice que las pruebas de historia se completan cuando todas las pruebas relevantes para la historia pasan o se cumple el nivel de automatización de pruebas.

Pruebas ágiles: herramientas

En proyectos ágiles, los probadores son responsables de las siguientes tareas diarias:

- Apoyar a los desarrolladores en la codificación, con aclaraciones sobre el comportamiento esperado del sistema.
- Ayude a los desarrolladores a crear pruebas unitarias efectivas y eficientes.
- Desarrollar scripts de automatización.
- Integre herramientas / scripts de pruebas de automatización con integración continua para pruebas de regresión.

Para una implementación rápida y eficaz de estas tareas, en la mayoría de los proyectos Agile se utiliza un sistema de Integración Continua (CI) que admite CI de Código y componentes de prueba.

Los probadores y los desarrolladores en proyectos ágiles pueden beneficiarse de varias herramientas para administrar las sesiones de

prueba y crear y enviar informes de defectos. Además de las herramientas especializadas para pruebas ágiles, los equipos ágiles también pueden beneficiarse de la automatización de pruebas y las herramientas de gestión de pruebas.

Nota : las soluciones de grabación y reproducción, última prueba, peso pesado y automatización de pruebas no son tan ágiles como:

- El flujo de trabajo de prueba final alentado por tales herramientas no funciona para equipos ágiles.
- Los scripts que no se pueden mantener creados con tales herramientas se convierten en un impedimento para el cambio.
- Estas herramientas especializadas crean la necesidad de especialistas en automatización de pruebas y, por lo tanto, fomentan los silos

Las herramientas que se utilizan ampliamente son:

S. No	Herramienta y propósito
1	Hudson Marco de CI
2	Selenio Pruebas funcionales: integradas con Hudson
3	CruiseControl Marco de CI
4	Junit

	Prueba unitaria de Java
5	Nunit Prueba de unidad .Net
6	Cobertura / JavaCodeCoverage / JFeature / JCover / Cobertura de prueba de Java
7	Bufón Java - Prueba de mutación / Sembrado automático de errores
8	Gretel Herramienta de monitoreo de cobertura de prueba de Java
9	TestCocoon C / C ++ o C #: reduce la cantidad de pruebas al encontrar pruebas redundantes y encuentra código muerto
10	JAZZ Java: cobertura de ramas, nodos y desactivación e implementa una GUI, planificadores de pruebas, instrumentación dinámica y un analizador de pruebas
11	Hormiga Java - Compilación de automatización
12	Nant .Net - Compilación de automatización
13	Hoguera

Herramientas de automatización de pruebas ágiles

Soporte eficaz de herramientas de automatización de pruebas ágiles

-

- Automatización temprana de pruebas utilizando un enfoque de prueba primero.
- Escribir código de automatización de pruebas utilizando lenguajes reales, lenguajes específicos de dominio.
- Centrándose en el comportamiento esperado del sistema.
- Separar la esencia de la prueba de los detalles de implementación, lo que la hace independiente de la tecnología.
- Fomento de la colaboración.

Las pruebas unitarias automatizadas (utilizando Junit o NUnit) admiten el enfoque de prueba primero para la codificación. Estas son pruebas de caja blanca y garantizan que el diseño sea sólido y que no haya defectos. Estas pruebas las crean los desarrolladores con el apoyo de los evaluadores y pueden ser independientes de la funcionalidad requerida. Esto da como resultado la entrega de un producto que puede no cumplir con los requisitos del cliente y, por lo tanto, sin valor comercial.

Esta preocupación se aborda automatizando las Pruebas de aceptación que se escriben con la colaboración del cliente, otras partes interesadas, evaluadores y desarrolladores. Las Pruebas de aceptación automatizadas son escritas por los clientes o propietarios de productos / analistas comerciales que reflejan el comportamiento

esperado del producto. La participación de los desarrolladores asegura la producción de código según los requisitos. Sin embargo, si la prueba se centra solo en la aceptación, el código resultante puede permanecer no extensible.

Por lo tanto, las pruebas unitarias automatizadas y las pruebas de aceptación automatizadas son complementarias y ambas son necesarias en el desarrollo ágil.

Las herramientas y marcos ágiles que admiten las pruebas de aceptación automatizadas son:

- Encajar
- Fitnesse
- Concordion
- Rubí
- Pepino

Encajar

Ward Cunningham desarrolló la herramienta Fit que se puede utilizar para la automatización de pruebas de aceptación. El ajuste permite -

- Clientes o propietarios de productos para dar ejemplos del comportamiento del producto utilizando Microsoft Word y Microsoft Excel.
- Programadores para convertir fácilmente esos ejemplos en pruebas automatizadas.

Fit 1.1 es compatible con Java y .NET.

FitNesse

FitNesse es una wiki, que es un estilo de servidor web que permite a cualquier visitante realizar modificaciones, incluido el cambio de páginas existentes y la creación de nuevas páginas. Un lenguaje de marcado simple le permite crear fácilmente títulos, poner texto en

negrita, subrayado y cursiva, crear listas con viñetas y realizar otros tipos de formateo simple.

En FitNesse, la automatización de la prueba de aceptación es la siguiente:

- Exprese las pruebas como tablas de datos de entrada y datos de salida esperados.
- Utilice FitNesse para colocar la tabla de prueba en la página que puede editar.
 - Alternativamente, coloque la tabla de prueba en Microsoft Excel, **cópiela** al portapapeles y luego use el comando **Spreadsheet to FitNesse** para que FitNesse formatee su tabla correctamente
- Ejecutar la prueba
- Obtiene los resultados de la prueba mediante la codificación de colores de las celdas en la tabla de prueba
 - las celdas verdes representan que se obtienen los valores esperados
 - Los glóbulos rojos representan que se obtiene un valor diferente al esperado
 - las celdas amarillas representan que se lanzó una excepción

Pepino

Cucumber es una herramienta basada en el marco de desarrollo impulsado por el comportamiento (BDD). Las características clave

son:

- Se utiliza para escribir pruebas de aceptación para aplicaciones web.
- Permite la automatización de la validación funcional en un formato fácilmente legible y comprensible como el inglés simple.
- Se implementó en Ruby y luego se extendió al marco de Java. Ambos apoyan a Junit.
- Admite otros lenguajes como Perl, PHP, Python, .Net, etc.
- Se puede utilizar junto con selenio, watir, carpincho, etc.